# Code Explanation

# Code Explanation: AWS Glue PySpark ETL JobThis code implements a complete ETL (Extract, Transform, Load) pipeline for processing customer and order data using AWS Glue, PySpark, and Apache Hudi with SCD Type 2 (Slowly Changing Dimension) support.---

## High-Level Flow

1. **Initialize** Spark/Glue contexts and load configuration2. **Ingest** customer and order data from S3 (CSV format)3. **Clean** data by removing nulls and duplicates4. **Catalog** cleaned data as Parquet tables in AWS Glue Data Catalog5. **Join** customer and order datasets6. **Enrich** with SCD Type 2 metadata columns7. **Write** final dataset to Apache Hudi table for historical tracking---

## Detailed Step-by-Step Breakdown

## **Configuration Loading (`get_job_parameters`)**

- Attempts to load parameters from a YAML configuration file- Falls back to hardcoded defaults if file not found- Returns dictionary containing: - Input/output S3 paths - File formats (CSV, Parquet, Hudi) - Catalog database and table names - Hudi-specific settings (record keys, partition fields, table type) - CSV parsing options

## **Context Initialization (`initialize_spark_contexts`)**

- Detects if running in AWS Glue environment or locally- Creates SparkContext, GlueContext, SparkSession, and Job objects- Initializes the Glue job with a name

## **S3 Access Validation (`validate_s3_access`)**

- Parses S3 path to extract bucket and prefix- Uses boto3 to verify path accessibility via `list_objects_v2`- Returns True on success or if validation fails (allows job to proceed)---

## Helper Classes

## **DataReader Class**

Encapsulates data reading logic with error handling:- **`_read_csv`**: Internal method to read CSV files with configurable options (header, delimiter, schema inference)- **`read_data_safe`**: Public method that: - Validates S3 access - Routes to appropriate reader based on format (CSV, Parquet, or generic) - Returns DataFrame or None on failure - Logs success/error messages

## **DataWriter Class**

Encapsulates data writing logic with error handling:- **`_write_parquet`**: Writes DataFrame to Parquet format- **`_write_csv`**: Writes DataFrame to CSV format with header option- **`_write_hudi`**: Writes DataFrame to Hudi table with custom options- **`write_data_safe`**: Public method that: - Validates S3 access - Routes to appropriate writer based on format - Returns True/False based on success - Logs success/error messages---

## Data Transformation Functions

### **`normalize_columns`**

- Converts all DataFrame column names to lowercase- Ensures consistent column naming across datasets

### **`remove_nulls`**

- Removes rows with actual NULL values using `dropna()`- Filters out rows containing string literals: 'Null', 'null', 'NULL'- Returns cleaned DataFrame

### **`remove_duplicates`**

- Uses `dropDuplicates()` to remove duplicate rows based on all columns- Returns deduplicated DataFrame

### **`add_scd_columns`**

Adds SCD Type 2 tracking columns:- **IsActive**: Boolean flag (True for current records)- **StartDate**: Timestamp when record became active- **EndDate**: Timestamp when record became inactive (NULL for active records)- **OpTs**: Operation timestamp for Hudi precombine logic---

## Catalog and Hudi Functions

### **`register_catalog_table`**

- Creates database in Glue Data Catalog if it doesn't exist- Writes DataFrame to S3 as Parquet- Registers table in catalog using `saveAsTable()`- Enables querying via Athena or other tools

### **`write_hudi_table`**

Configures and writes Hudi table with:- **Record key**: Unique identifier (OrderId)- **Precombine key**: Timestamp for conflict resolution (OpTs)- **Partition field**: Data partitioning strategy (Region)- **Table type**: COPY_ON_WRITE (optimized for read-heavy workloads)- **Operation**: Upsert (insert or update based on record key)- **Hive sync**: Automatic catalog synchronization---

## Main ETL Pipeline (`main` function)

### **Step 1: Ingest Customer Data**

- Reads customer CSV from S3 using configured path- Normalizes column names to lowercase- Logs record count

### **Step 2: Ingest Order Data**

- Reads order CSV from S3 using configured path- Normalizes column names to lowercase- Logs record count

## **Step 3: Clean Customer Data**

- Removes NULL values and 'Null' string literals- Removes duplicate records- Logs record counts after each operation

## **Step 4: Clean Order Data**

- Removes NULL values and 'Null' string literals- Removes duplicate records- Logs record counts after each operation

## **Step 5: Catalog Customer Data**

- Writes cleaned customer data to S3 as Parquet- Registers table in Glue Data Catalog- Enables downstream querying

## **Step 6: Catalog Order Data**

- Writes cleaned order data to S3 as Parquet- Registers table in Glue Data Catalog- Enables downstream querying

## **Step 7: Join Datasets**

- Performs inner join on `custid` column- Validates that join key exists in both DataFrames- Logs joined record count

## **Step 8: Add SCD Type 2 Columns**

- Enriches joined data with tracking metadata- Prepares data for historical change tracking

## **Step 9: Write to Hudi**

- Writes final dataset to Hudi table- Enables upsert operations for future updates- Automatically syncs with Glue Data Catalog

## **Finalization**

- Commits Glue job- Logs completion status---

## Key Design Patterns

1. **Error Resilience**: All I/O operations wrapped in try-except with logging2. **Configuration Externalization**: Parameters loaded from YAML (not hardcoded)3. **Separation of Concerns**: Reader/Writer classes isolate I/O logic4. **Validation**: S3 access checked before operations5. **Traceability**: Extensive logging at each pipeline stage6. **SCD Type 2**: Historical tracking via IsActive, StartDate, EndDate, OpTs7. **Idempotency**: Upsert operations allow safe re-runs---

## Security Note

The code uses placeholder patterns for S3 paths (e.g., `s3://<BUCKET_NAME>/path/`) in production deployments. Actual bucket names and account IDs should be externalized via AWS Secrets Manager, Parameter Store, or environment variables.